

Task -9 : Employee Management Application using MVC Architecture

Aim : To develop an employee management application using MVC architecture with HTML, CSS, JavaScript, Node.js, and SQLite.

The system simulates Spring MVC concepts by separating controller, model, and view using annotation-like modular design.

Algorithm :

1. Start the Node.js server to handle client requests.
2. Establish a connection to the SQLite database.
3. Create a table to store employee details (id, name, role, salary).
4. Design the user interface using HTML and CSS.
5. Accept employee details from the user through input fields.
6. Send data from the frontend to backend using fetch API.
7. Implement controller logic using Express routes to handle requests.
8. Insert employee data into the database through the model layer.
9. Create an API to retrieve employee records from the database.
10. Fetch employee data and send it back to the frontend.
11. Display employee details dynamically in the UI.
12. Maintain separation of concerns between controller, model, and view components.

Program :

Index.html :

```
<!DOCTYPE html>

<html>

<head>

  <title>Spring MVC Style App</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<div class="container">
```

```
<h1> 🧑 Employee MVC App</h1>
```

```
<input type="text" id="name" placeholder="Name">
```

```
<input type="text" id="role" placeholder="Role">
```

```
<input type="number" id="salary" placeholder="Salary">
```

```
<button onclick="addEmployee()">Add Employee</button>
```

```
<button onclick="viewEmployees()">View Employees</button>
```

```
<div id="output"></div>
```

```
</div>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Style.css :

```
body {
```

```
    font-family: Arial;
```

```
    background: linear-gradient(135deg, #667eea, #764ba2);
```

```
    display: flex;
```

```
    justify-content: center;
```

```
    align-items: center;
```

```
    height: 100vh;
```

```
}
```

```
.container {
```

```
background: white;
padding: 30px;
border-radius: 15px;
width: 350px;
text-align: center;
}
```

```
input {
  width: 90%;
  padding: 10px;
  margin: 8px 0;
}
```

```
button {
  padding: 10px;
  margin: 5px;
  background: #00c9a7;
  color: white;
  border: none;
}
```

Server.js :

```
const express = require("express");
const sqlite3 = require("sqlite3").verbose();
const cors = require("cors");
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.use(cors());
```

```
app.use(express.static(__dirname));
```

```
const db = new sqlite3.Database("./employee.db");
```

```
// Model (DB)
```

```
db.run(`
```

```
CREATE TABLE IF NOT EXISTS employees (
```

```
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  name TEXT,
```

```
  role TEXT,
```

```
  salary INTEGER
```

```
)
```

```
`);
```

```
// Controller - Add employee
```

```
app.post("/add", (req, res) => {
```

```
  const { name, role, salary } = req.body;
```

```
  db.run(
```

```
    "INSERT INTO employees (name, role, salary) VALUES (?, ?, ?)",
```

```
    [name, role, salary],
```

```
    () => res.json({ message: "Employee Added  " })
```

```
  );
```

```
});
```

```
// Controller - Get employee
```

```
app.get("/employees", (req, res) => {
```

```
  db.all("SELECT * FROM employees", [], (err, rows) => {
```

```
    res.json(rows);
```

```
  });
```

```
});
```

```
app.listen(3000, () => {  
  console.log("🚀 Server running at http://localhost:3000");  
});
```

Script.js :

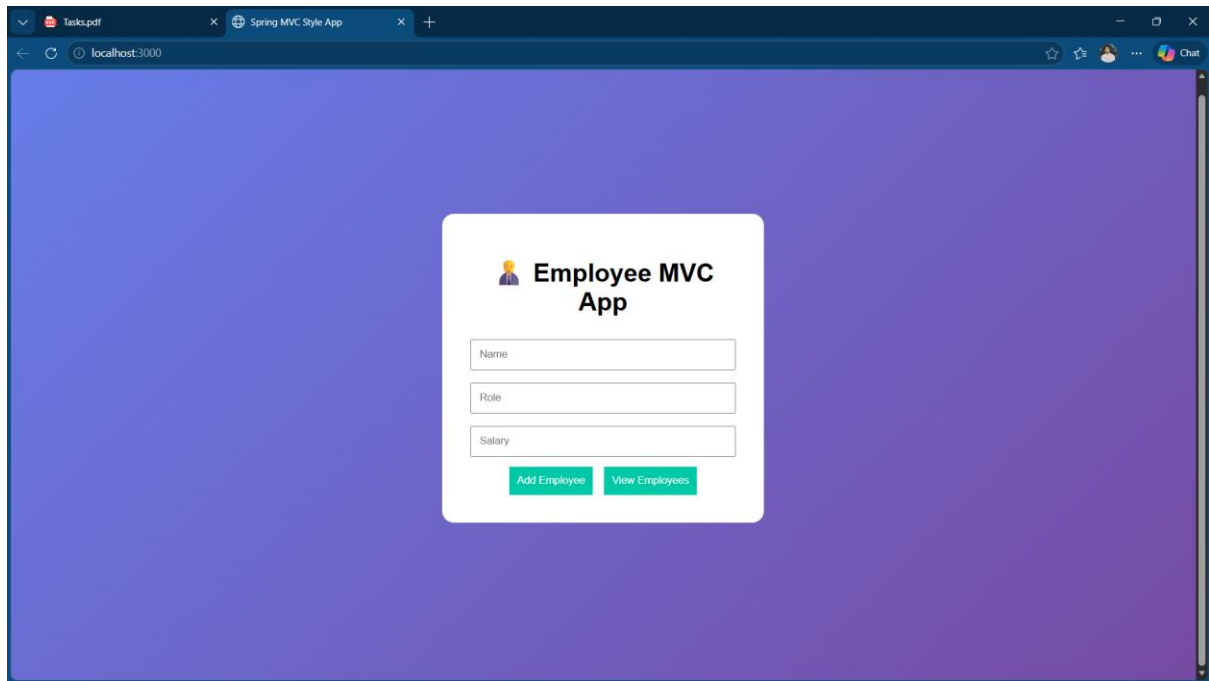
```
async function addEmployee() {  
  const name = document.getElementById("name").value;  
  const role = document.getElementById("role").value;  
  const salary = document.getElementById("salary").value;  
  
  await fetch("/add", {  
    method: "POST",  
    headers: {  
      "Content-Type": "application/json"  
    },  
    body: JSON.stringify({ name, role, salary })  
  });  
  
  alert("Employee Added");  
}
```

```
async function viewEmployees() {  
  const res = await fetch("/employees");  
  const data = await res.json();  
  
  let html = "<h3>Employees</h3>";  
  
  data.forEach(emp => {
```

```
html += `<p>${emp.name} - ${emp.role} - ₹${emp.salary}</p>`;
});
```

```
document.getElementById("output").innerHTML = html;
}
```

Output :



Result : The system successfully demonstrates the MVC architecture by managing employee data through structured separation of controller, model, and view components.